

Университет ИТМО

Факультет Программной Инженерии и Компьютерной техники

Программирование

Лабораторная работа №2

Вариант № 1449

Выполнил:

Крахмальников Илья Артемович

Группа Р3119

Преподаватели:

Бойко В. А.

Гаврилов А. В.

Санкт-Петербург

2026

Содержание

Задание	3
Основные этапы решения	4
Реализация покемона Volbeat.....	4
Реализация атаки QuickAttack для Volbeat	5
Реализация остальных 3 классов атак Volbeat	5
Реализация остальных 5 покемонов	7
Реализация цепочек эволюции.....	8
Реализация боя	8
Результат программы.....	9
Исходный код	13
Диаграмма классов.....	13
Выводы по работе	14
Список литературы	15

Задание

Ваши покемоны:

 Volbeat Атаки: Quick Attack Flash Aerial Ace Dizzy Punch	 Spoink Атаки: Charge Beam Calm Mind Thunder Wave	 Grumpig Атаки: Charge Beam Calm Mind Thunder Wave Focus Blast	 Poliwag Атаки: Double Team Blizzard	 Poliwhirl Атаки: Double Team Blizzard Belly Drum	 Poliwrath Атаки: Double Team Blizzard Belly Drum Bulk Up
---	---	--	---	---	---

На основе базового класса **Pokemon** написать свои классы для заданных видов покемонов. Каждый вид покемона должен иметь один или два типа и стандартные базовые характеристики:

- очки здоровья (HP)
- атака (attack)
- защита (defense)
- специальная атака (special attack)
- специальная защита (special defense)
- скорость (speed)

Классы покемонов должны наследоваться в соответствии с цепочкой эволюции покемонов. На основе базовых классов **PhysicalMove**, **SpecialMove** и **StatusMove** реализовать свои классы для заданных видов атак. Все разработанные классы, не имеющие наследников, должны быть реализованы таким образом, чтобы от них нельзя было наследоваться.

Атака должна иметь стандартные тип, силу (power) и точность (accuracy). Должны быть реализованы стандартные эффекты атаки. Назначить каждому виду покемонов атаки в соответствии с вариантом. Уровень покемона выбирается минимально необходимым для всех реализованных атак.

Используя класс симуляции боя **Battle**, создать 2 команды покемонов (каждый покемон должен иметь имя) и запустить бой.

Базовые классы и симулятор сражения находятся в [jar-архиве](#) (обновлен 9.10.2018, исправлен баг с добавлением атак и кодировкой). Документация в формате javadoc - [здесь](#).

Информацию о покемонах, цепочках эволюции и атаках можно найти на сайтах <http://poke-universe.ru>, <http://pokemondb.net>, <http://veekun.com/dex/pokemon>

Комментарии

Цель работы: на простом примере разобраться с основными концепциями ООП и научиться использовать их в программах.

Что надо сделать (краткое описание)

1. Ознакомиться с [документацией](#), обращая особое внимание на классы **Pokemon** и **Move**. При дальнейшем выполнении лабораторной работы читать документацию еще несколько раз.
2. Скачать файл **Pokemon.jar**. Его необходимо будет использовать как для компиляции, так и для запуска программы. Распаковывать его не надо! Нужно научиться подключать внешние jar-файлы к своей программе.
3. Написать минимально работающую программу и посмотреть как она работает.

```
Battle b = new Battle();
Pokemon p1 = new Pokemon("Чужой", 1);
Pokemon p2 = new Pokemon("Хищник", 1);
b.addAlly(p1);
b.addFoe(p2);
b.go();
```

4. Создать один из классов покемонов для своего варианта. Класс должен наследоваться от базового класса **Pokemon**. В конструкторе нужно будет задать типы покемона и его базовые характеристики. После этого попробуйте добавить покемона в сражение.
5. Создать один из классов атак для своего варианта (лучше всего начать с физической или специальной атаки). Класс должен наследоваться от класса **PhysicalMove** или **SpecialMove**. В конструкторе нужно будет задать тип атаки, ее силу и точность. После этого добавить атаку покемону и проверить ее действие в сражении. Не забудьте переопределить метод **describe**, чтобы выводилось нужное сообщение.
6. Если действие атаки отличается от стандартного, например, покемон не промахивается, либо атакующий покемон также получает повреждение, то в классе атаки нужно дополнительно переопределить соответствующие методы (см. документацию). При реализации атак, которые меняют статус покемона (наследники **StatusMove**), скорее всего придется разобраться с классом **Effect**. Он позволяет на один или несколько ходов изменить состояние покемона или модификатор его базовых характеристик.
7. Доделать все необходимые атаки и всех покемонов, распределить покемонов по командам, запустить сражение.

Отчёт по работе должен содержать:

1. Текст задания.
2. Диаграмма классов реализованной объектной модели.
3. Исходный код программы.
4. Результат работы программы.
5. Выводы по работе.

Вопросы к защите лабораторной работы:

1. Объектно-ориентированное программирование. Основные понятия: объекты, наследование, полиморфизм, инкапсуляция.
2. Понятие класса. Классы и объекты в Java.
3. Члены класса. Поля, методы, конструкторы. Модификаторы доступа.
4. Создание и инициализация объектов. Вызовы методов.
5. Области видимости переменных.
6. Модификаторы **final** и **static**.
7. Пакеты, инструкция **import**.

Основные этапы решения

Реализация покемона Volbeat

Нашел информацию про покемона на сайте pokemondb.net

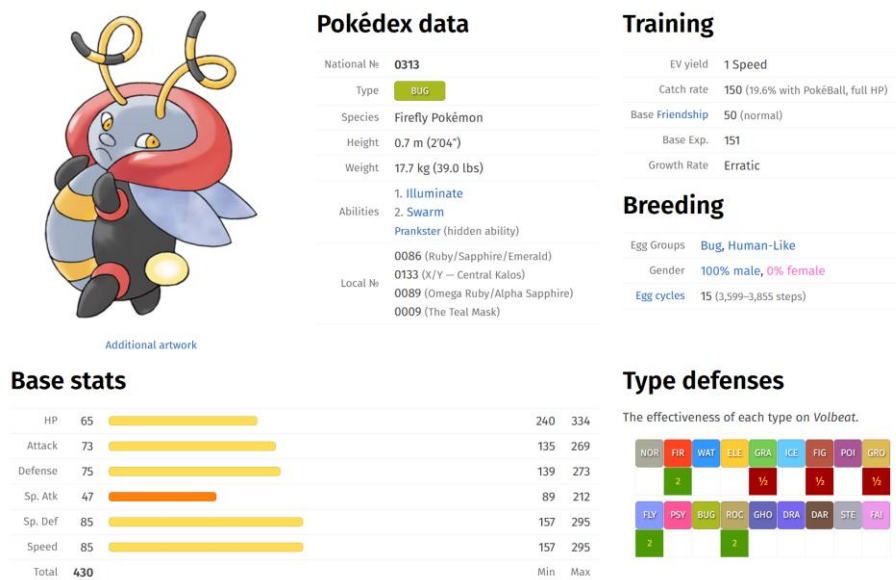


Рисунок 1 – информация про Volbeat

На основе этой информации создал класс Volbeat и задал в конструкторе его базовые характеристики

```
package pokemon;

import move.*;
import ru.ifmo.se.pokemon.*;

public class Volbeat extends Pokemon { no usage
    public Volbeat(String name, int level) {
        super(name, level);
        setType(Type.BUG);
        setStats(65, 73, 74, 47, 85, 85);
    }
}
```

Рисунок 1.2 – Реализация Volbeat

Реализация атаки QuickAttack для Volbeat

Для покемона требуется создать 4 вида атак, одна из них - QuickAttack

С того же сайта беру информацию про данную атаку и на ее основе создаю класс QuickAttack

Quick Attack (move)	
Contents Flavor text Learnt by level up Learnt by breeding	
Move data	Effects
Type NORMAL	<i>Quick Attack</i> deals damage and has a priority of +1.
Category  Physical	Moves with a higher priority always go first, regardless of a Pokémon's speed. If two moves with the same priority are used, the faster Pokémon goes first as usual.
Power 40	
Accuracy 100	
PP 30 (max. 48)	
Priority +1	
Makes contact? Yes	
Introduced Generation 1	
	Z-Move effects
	When a Pokémon is holding Normalium Z and uses its Z-Power, <i>Quick Attack</i> turns into Breakneck Blitz and has base power 100.

Рисунок 2.1 – информация про Quick Attack

```
package move;

import ru.ifmo.se.pokemon.*;

public class QuickAttack extends PhysicalMove {
    public QuickAttack() { 1 usage
        super(Type.NORMAL, 40, 100, 1, 1);
    }

    @Override no usages
    public String describe() {
        return "использует Quick Attack";
    }
}
```

Рисунок 2.2 – реализация QuickAttack

Также я переопределил метод describe(), который возвращает описание атаки

Реализация остальных 3 классов атак Volbeat

Далее я создал класс Flash, который наследуется от StatusMove

В нем было необходимо переопределить метод `applyOppEffects(Pokemon p)`, поскольку атака снижает у защищающегося покемона точность атаки

```
package move;

import ru.ifmo.se.pokemon.*;

public class Flash extends StatusMove { 1 usage
    public Flash() { 1 usage
        super(Type.NORMAL, 0, 100);
    }

    @Override no usages
    protected void applyOppEffects(Pokemon p) {
        p.setMod(Stat.ACCURACY, -1);
    }

    @Override no usages
    public String describe() { return "использует Quick Attack"; }
}
```

Рисунок 3.1 – реализация Flash

У атаки Aerial Ace необходимо было сделать точность 100%, поэтому я переопределил метод `checkAccuracy(Pokemon att, Pokemon def)`, который теперь всегда возвращает `true`, то есть всегда попадает

```
package move;

import ru.ifmo.se.pokemon.*;

public class AerialAce extends PhysicalMove { 1 usage
    public AerialAce() { 1 usage
        super(Type.FLYING, 60, 100);
    }

    @Override no usages
    public boolean checkAccuracy(Pokemon att, Pokemon def) {
        return true;
    }

    @Override no usages
    public String describe() {
        return "использует Aerial Ace";
    }
}
```

Рисунок 3.2 – реализация AerialAce

Далее я реализовал атаку Dizzy Punch, особенность которой - это наложение эффекта растерянности

Для такого наложения такого эффекта я использовал `Effect.confuse()`; а 20% шанс проверял через `Math.random()`

```

package move;

import ru.ifmo.se.pokemon.*;

public class DizzyPunch extends PhysicalMove {
    public DizzyPunch() {
        super(Type.NORMAL, 70, 100);
    }

    @Override
    public void applyOppEffects(Pokemon p) {
        if (Math.random() < 0.2) {
            Effect.confuse(p);
        }
    }

    @Override
    public String describe() {
        return "использует Dizzy Punch";
    }
}

```

Рисунок 3.3 – реализация DizzyPunch

Реализация остальных 5 покемонов

Аналогично Volbeat я создал оставшихся покемонов и их атаки

Среди новых методов, которые я переопределил – это `applySelfEffects` у атаки `ChargeBeam`, наложение эффекта на атакующего покемона, с 70% шансом увеличение ступени специальной атаки

```

@Override
public void applySelfEffects(Pokemon p) {
    Effect e = new Effect().chance(0.7).stat(Stat.SPECIAL_ATTACK, +1);
    p.addEffect(e);
}

```

Рисунок 4.1 – реализация `applySelfEffects` у атаки `ChargeBeam`

У атаки `Calm Mind` для увеличения характеристики специальной атаки и защиты использовал метод `setMod()`

```
@Override no usages
public void applySelfEffects(Pokemon p) {
    p.setMod(Stat.SPECIAL_ATTACK, +1);
    p.setMod(Stat.SPECIAL_DEFENSE, +1);
}
```

Рисунок 4.2 – реализация applySelfEffects у атаки CalmMind

При атаке BellyDrum необходимо атакующему снять половину хп от максимального значения, для этого использовал метод setMod, в который передал Stat.HP и значение хп, которое надо уменьшить (вычислил через p.getStat(Stat.HP))

```
@Override no usages
protected void applySelfEffects(Pokemon p) {
    int damage = (int)(p.getStat(Stat.HP) / 2);
    p.setMod(Stat.HP, damage); // умрет, если меньше половины хп (походу так должно быть)
    p.setMod(Stat.ATTACK, +6);
}
```

Рисунок 4.3 – реализация applySelfEffects у атаки BellyDrum

Реализация цепочек эволюции

Поскольку покемон Grumpig является эволюцией покемона Spoink, а также есть цепочка эволюции Poliwhag -> Poliwhirl -> Poliwrath, то поэтому я реализовал цепочку наследований между классами

Финальной эволюции класса дал модификатор final, чтобы от него уже нельзя было наследоваться

```
public class Poliwhag extends Pokemon
public class Poliwhirl extends Poliwhag
public final class Poliwrath extends Poliwhirl
```

Реализация боя

Для симуляции боя я создал объект Battle и покемонов, каждого по 1 штуке из ранее созданных классов покемонов, распределил покемонов по группам и запустил сражение

```

import pokemon.*;
import ru.ifmo.se.pokemon.*;

public class Main {
    public static void main(String[] args) {
        Battle b = new Battle();

        Pokemon p1 = new Volbeat( name: "Пчелка", level: 10);
        Pokemon p2 = new Spoink( name: "Попрыгунчик", level: 10);
        Pokemon p3 = new Grumpig( name: "Крот", level: 10);

        Pokemon e1 = new Poliwhag( name: "Рыбка", level: 10);
        Pokemon e2 = new Poliwhirl( name: "Друган", level: 10);
        Pokemon e3 = new Poliwrath( name: "Гигант", level: 10);

        b.addAlly(p1);
        b.addAlly(p2);
        b.addAlly(p3);

        b.addFoe(e1);
        b.addFoe(e2);
        b.addFoe(e3);

        b.go();
    }
}

```

Рисунок 5 - реализация боя

Результат программы

Volbeat Пчелка из команды фиолетовых вступает в бой!

Poliwhag Рыбка из команды белых вступает в бой!

Poliwhag Рыбка промахивается

Volbeat Пчелка использует Aerial Ace.

Критический удар!

Poliwhag Рыбка теряет 46 здоровья.

Poliwhag Рыбка использует Blizzard.

Volbeat Пчелка теряет 21 здоровья.

Volbeat Пчелка использует Flash.

Poliwhag Рыбка уменьшает точность.

Poliwhag Рыбка промахивается

Volbeat Пчелка использует Aerial Ace.

Poliwhag Рыбка теряет 33 здоровья.

Volbeat Пчелка использует Quick Attack.

Poliwhag Рыбка теряет 18 здоровья.

Poliwag Рыбка использует Blizzard.
Volbeat Пчелка теряет 17 здоровья.

Poliwag Рыбка борется с соперником.
Volbeat Пчелка теряет 17 здоровья.
Poliwag Рыбка теряет 4 здоровья.

Volbeat Пчелка использует Aerial Ace.
Poliwag Рыбка теряет 23 здоровья.

Volbeat Пчелка использует Quick Attack.
Критический удар!
Poliwag Рыбка теряет 44 здоровья.
Poliwag Рыбка теряет сознание.
Poliwhirl Друган из команды белых вступает в бой!
Poliwhirl Друган использует Blizzard.
Volbeat Пчелка теряет 24 здоровья.

Volbeat Пчелка использует Flash.
Poliwhirl Друган уменьшает точность.

Volbeat Пчелка использует Quick Attack.
Критический удар!
Poliwhirl Друган теряет 28 здоровья.

Poliwhirl Друган промахивается

Poliwhirl Друган промахивается

Volbeat Пчелка использует Dizzy Punch.
Poliwhirl Друган теряет 19 здоровья.

Volbeat Пчелка использует Quick Attack.
Критический удар!
Poliwhirl Друган теряет 22 здоровья.

Poliwhirl Друган использует Blizzard.
Критический удар!
Volbeat Пчелка теряет 51 здоровья.

Poliwhirl Друган промахивается

Volbeat Пчелка использует Aerial Ace.
Критический удар!
Poliwhirl Друган теряет 35 здоровья.

Poliwhirl Друган использует Blizzard.
Volbeat Пчелка теряет 25 здоровья.

Volbeat Пчелка использует Dizzy Punch.
Критический удар!
Poliwhirl Друган теряет 50 здоровья.

Poliwhirl Друган промахивается

Volbeat Пчелка использует Flash.
Poliwhirl Друган уменьшает точность.

Poliwhirl Друган промахивается

Volbeat Пчелка использует Flash.
Poliwhirl Друган уменьшает точность.

Volbeat Пчелка использует Quick Attack.
Poliwhirl Друган теряет 18 здоровья.

Poliwhirl Друган борется с соперником.
Volbeat Пчелка теряет 11 здоровья.
Poliwhirl Друган теряет 3 здоровья.

Volbeat Пчелка использует Quick Attack.
Poliwhirl Друган теряет 18 здоровья.
Poliwhirl Друган теряет сознание.
Poliwrath Гигант из команды белых вступает в бой!
Volbeat Пчелка использует Aerial Ace.
Poliwrath Гигант теряет 28 здоровья.

Poliwrath Гигант использует Blizzard.
Volbeat Пчелка теряет 38 здоровья.
Volbeat Пчелка теряет сознание.
Spoink Попрыгунчик из команды фиолетовых вступает в бой!
Poliwrath Гигант промахивается

Spoink Попрыгунчик промахивается

Poliwrath Гигант использует Blizzard.
Критический удар!
Spoink Попрыгунчик теряет 62 здоровья.

Spoink Попрыгунчик промахивается

Poliwrath Гигант промахивается

Spoink Попрыгунчик промахивается

Poliwrath Гигант промахивается

Spoink Попрыгунчик использует Charge Beam.
Poliwrath Гигант теряет 27 здоровья.

Poliwrath Гигант промахивается

Spoink Попрыгунчик использует Thunder Wave.
Poliwrath Гигант парализован

Poliwrath Гигант промахивается

Spoink Попрыгунчик использует Charge Beam.
Poliwrath Гигант теряет 45 здоровья.

Poliwrath Гигант промахивается

Spoink Попрыгунчик использует Charge Beam.
Poliwrath Гигант теряет 36 здоровья.

Poliwrath Гигант промахивается

Spoink Попрыгунчик использует Thunder Wave.

Poliwrath Гигант использует Blizzard.
Критический удар!
Spoink Попрыгунчик теряет 46 здоровья.

Spoink Попрыгунчик использует Thunder Wave.

Spoink Попрыгунчик промахивается

Poliwrath Гигант промахивается

Spoink Попрыгунчик использует Charge Beam.
Poliwrath Гигант теряет 41 здоровья.

Poliwrath Гигант промахивается

Команда фиолетовых побеждает в этом бою!

Выводы по работе

1. Изучил принципы ООП в java, научился переопределять методы
2. Научился проектировать диаграмму класса
3. Разобрался в модификаторах public, protected, static, final
4. Научился импортировать классы из jar архива

Список литературы

1. Pokémon Database - <https://pokemondb.net/> (дата обращения: 27.09.2026)
2. Документация ru.ifmo.se.pokemon - <https://se.ifmo.ru/~tony/doc/> (дата обращения: 27.09.2026)